

Laboratorio di Calcolo

A.A. 2026/2027, Canale A-Dec

Giacomo Artoni, Lorenzo Caprini

Numeri reali e virgola mobile

Rappresentazione dei Reali

I numeri reali $\mathbb{R}$ sono **infiniti** e continui. In un calcolatore dobbiamo memorizzarli su un numero **finito** di bit.

Opzione n.1: Virgola Fissa (Fixed-Point)

- Un numero fisso di bit per la parte intera e per la parte frazionaria.
- La posizione della virgola è **implicita** e bloccata a livello hardware.
- **Bit Interi (I):** determinano l'intervallo massimo (range).
- **Bit Frazionari (F):** determinano la risoluzione (2^{-F}).
- **Risoluzione costante:** lo spazio tra due numeri rappresentabili consecutivi è sempre fisso ($\Delta = 2^{-F}$).

Esempio pratico (8 bit: 4 int / 4 frac)

$0110.1010_2 = 4 + 2 + (2^{-1} + 2^{-3}) = 6.625_{10}$

- Risoluzione minima: $2^{-4} = 0.0625$
- Valore massimo: $1111.1111_2 = 15.9375$

Rappresentazione dei Reali

I numeri reali $\mathbb{R}$ sono **infiniti** e continui. In un calcolatore dobbiamo memorizzarli su un numero **finito** di bit.

Opzione n.2: Virgola Mobile (Floating-Point)

- Ogni numero N viene espresso in notazione scientifica:

$$N = (-1)^S \times C \times 2^E$$

- La virgola "scivola" mantenendo costante la precisione relativa, indipendentemente dall'ordine di grandezza.
- **Vantaggio fondamentale:** stessa struttura di memoria per rappresentare sia la massa dell'elettrone ($\approx 10^{-30}$ kg) che la massa del Sole ($\approx 10^{+30}$ kg).
- **Risoluzione variabile:** solo la precisione relativa rimane (circa) costante.

```
// Numero piccolo
+0.000001011₂
→ +1.011₂ × 2⁻⁶

// Numero grande
+101100000.0₂
→ +1.011₂ × 2⁺⁸
```

Notazione Normalizzata in Base 2:
Ogni numero reale non nullo è espresso come $\pm 1.F \times 2^E$, dove la parte intera è sempre 1 ed è **implicita** (bit nascosto o *hidden bit*).

Lo Standard IEEE 754

Lo standard universale adottato da tutte le moderne CPU per l'aritmetica in virgola mobile.

Valore Reale Rappresentato

$$V = (-1)^S \times (1 + M) \times 2^E$$

Il valore 1 prima della virgola è implicito (hidden bit).

- **Segno (S):** *1 bit* ($0 = \text{positivo}, 1 = \text{negativo}$).
- **Esponente (E):** memorizzato in *Eccesso* K (bias $K = 2^{n-1} - 1$).
- **Mantissa (M):** le cifre dopo la virgola ($1.M$).

Formati Principali

Singola Precisione: `float`

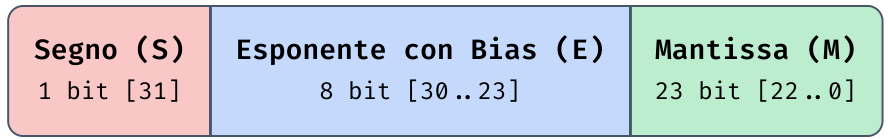
- Dimensione: **32 bit**
- Segno: **1 bit** | Esponente: **8 bit** | Mantissa: **23 bit**
- Bias: **127**

Doppia Precisione: `double`

- Dimensione: **64 bit**
- Segno: **1 bit** | Esponente: **11 bit** | Mantissa: **52 bit**
- Bias: **1023**

Struttura Bit IEEE 754 (32 bit)

Layout di memoria di un `float` a singola precisione:



Campo	Bit dedicati	Range valori	Note
Segno (S)	1	$0, 1$	$0 \to +, 1 \to -$
Esponente (E)	8	$1 \dots 254$	$E_{reale} = E - 127$ (0 e 255 riservati)
Frazione (F)	23	$0 \dots 2^{23} - 1$	$1.F$ fornisce ≈ 7 cifre decimali

Esempio di Conversione in IEEE 754

Convertiamo il numero reale $+13.625_{10}$ nel formato `float` (32 bit).

1. Conversione in virgola fissa

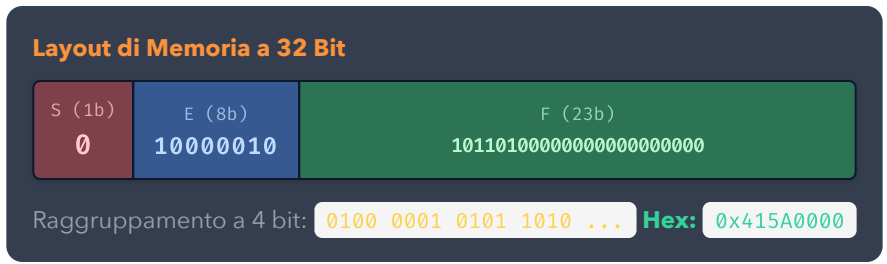
$$13.625_{10} \rightarrow 1101.101_2$$

2. Normalizzazione

Spostando la virgola a sinistra di 3 posizioni:

$$1101.101_2 \rightarrow 1.101101_2 \times 2^3$$

- **Esponente reale (e):** $+3$
  - Esponente (E): $e + \text{Bias} = 3 + 127 = 130_{10} = 10000010_2$
- **Mantissa (M):** `101101` (seguito da 17 zeri)



Valori Speciali e Denormalizzati

Le configurazioni estreme dell'esponente ($E = 0$ e $E = 255$) sono riservate a casi speciali:

Esponente (E)	Mantissa (F)	Valore Rappresentato	Significato / Utilizzo
00000000_2 **(0)**	$= 0$	± 0	Zero con segno ($+0$ e -0)
00000000_2 **(0)**	$\neq 0$	$(-1)^S \times (0.F) \times 2^{-126}$	**Numeri Denormalizzati** (gestione dell'underflow)
$1 \ldots 254$	Qualsiasi	$(-1)^S \times (1.F) \times 2^{E-127}$	**Numeri Normalizzati** (uso standard)
11111111_2 **(255)**	$= 0$	$\pm \infty$	Overflow / Divisione per zero
11111111_2 **(255)**	$\neq 0$	**NaN** (*Not a Number*)	Operazione non valida (es. $\sqrt{-1}, 0/0$)

Precisione Limitata

Non tutti i numeri decimali possono essere espressi in modo esatto in binario!

Il caso apparentemente semplice: 0.1_{10}

Convertiamo la parte frazionaria 0.1_{10} in binario:
(Attenzione! Invece di *dividere* per 2 e prendere il resto si moltiplica per 2 e si prende il resto)

- $0.1 \times 2 = 0.2 \rightarrow 0 \ \ \text{resto} \ \ 0.2$
- $0.2 \times 2 = 0.4 \rightarrow 0 \ \ \text{resto} \ \ 0.4$
- $0.4 \times 2 = 0.8 \rightarrow 0 \ \ \text{resto} \ \ 0.8$
- $0.8 \times 2 = 1.6 \rightarrow 1 \ \ \text{resto} \ \ 0.6$
- $0.6 \times 2 = 1.2 \rightarrow 1 \ \ \text{resto} \ \ 0.2\ldots$

$$0.1_{10} = 0.0001100110011\ldots_2 \quad (\text{periodico!})$$

Problemi di arrotondamento

1. L'inganno dell'uguaglianza (`==`)

Perché $0.1 + 0.2$ non fa mai 0.3 in memoria.

In `C`

```c
#include <stdio.h>

int main() {
    float a = 0.1f, b = 0.2f;
    if (a + b == 0.3f) {
        printf("Uguali!\n");
    } else {
        printf("Diversi! Valore: %.20f\n", a + b);
    }
    return 0;
}
// Output: Diversi! Valore: 0.30000001192092895508
```

In `python`

```python
a, b = 0.1, 0.2

print(a + b == 0.3)
# Output: False

print(a + b)
# Output: 0.30000000000000004
```

Problemi di arrotondamento

2. Non fidarsi delle proprietà matematiche

In algebra $(A + B) + C = A + (B + C)$. Nel calcolo numerico questo **non è sempre vero**.

In `C`

```c
#include <stdio.h>

int main() {
    double A = 1e8, B = -1e8, C = 1e-8;
    // Ordine 1: (A + B) + C
    printf("(A + B) + C = %e\n", (A + B) + C);
    // Output: (A + B) + C = 1.000000e-08

    // Ordine 2: A + (B + C)
    printf("A + (B + C) = %e\n", A + (B + C));
    // Output: A + (B + C) = 1.490116e-08
    return 0;
}
```

In `python`

```python
A, B, C = 1e8, -1e8, 1e-8

# Ordine 1: (A + B) + C
print((A + B) + C)
# Output: 1e-08  (CORRETTO)

# Ordine 2: A + (B + C)
print(A + (B + C))
# Output: 0.0    (ERRORE!)
```

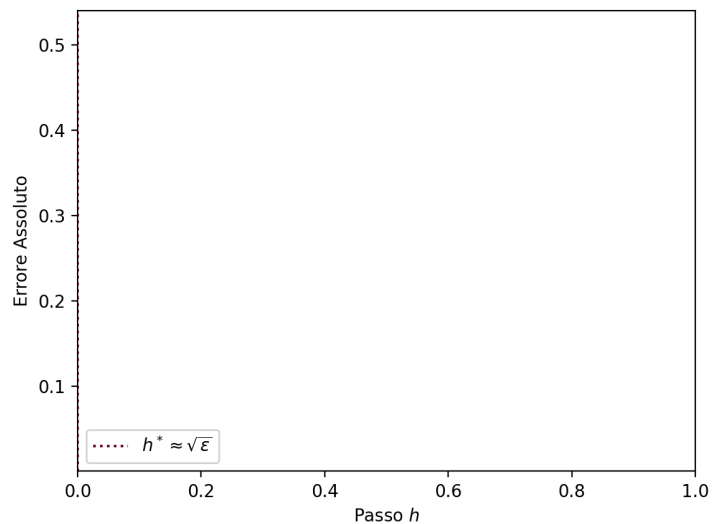
Problemi di arrotondamento

3. Attenzione alle sottrazioni!

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}$$

```
# Genera 200 valori di h da 10^-16 a 10^0
h = np.logspace(-16, 0, 200, dtype=np.float64)

# Derivata numerica di sin(x)
df_num = (np.sin(x + h) - np.sin(x)) / h
df_true = np.cos(x)
errore = np.abs(df_num - df_true)
```



Machine Epsilon e Cancellazione Numerica

Due concetti chiave per il calcolo scientifico e la simulazione di sistemi fisici.

Machine Epsilon (ϵ_{mach})

- È il più piccolo valore positivo per cui:

$$1.0 + \epsilon_{mach} \neq 1.0$$

- Definisce la **precisione relativa** dell'architettura:
  - Singola precisione (`float`):
    $\epsilon \approx 2^{-23} = 1.19 \times 10^{-7}$
  - Doppia precisione (`double`):
    $\epsilon \approx 2^{-52} = 2.22 \times 10^{-16}$

Cancellazione Catastrofica

Si verifica quando si sottraggono due numeri quasi uguali. Le cifre significative più importanti si annullano, amplificando l'errore di arrotondamento residuo.

```
1.23456789 -
1.23456700 =
0.00000089  <--- Si perdono quasi tutte
                 le cifre significative!
```

Confronto tra Formati Numerici

Tipo	Bit Totali	Bit Esponente	Bit Mantissa	Cifre Decimali	Range Approssimativo
`float`	32	8	23 (+1)	≈ 7	$10^{-38} \dots 10^{+38}$
`double`	64	11	52 (+1)	$\approx 15 - 16$	$10^{-308} \dots 10^{+308}$
`long double`	80/128	15	64/112	$\approx 19 - 34$	$10^{-4932} \dots 10^{+4932}$